Tennis Tournament Management System

Student: Hoza Violeta Maria

Group: 30434

Software Design – Assignment 1

Technical University of Cluj-Napoca

Contents

1. Project overview

The Tennis Tournament Management System is a comprehensive web application that aims to simplify the organization and management of tennis tournaments. The system provides role-based access for 3 main user categories: Administrator, Referee, and Player.

The application is built using a modern tech stack:

- Backend: Java with Spring Boot, following a layered architecture
- Frontend: React for a responsive and intuitive user interface
- Database: JPA and Hibernate
- Authentication: JWT-based security for secure user access.

The application implements several security mechanisms to protect user data and prevent unauthorized access:

- Password Hashing: All user passwords are securely hashed using PasswordEncoder from Spring Security. This ensures that raw passwords are never stored in the database.
- JWT Authentication: The app uses JSON Web Tokens (JWT) for stateless authentication, ensuring secure access to endpoints.
- Role-Based Access Control (RBAC): Access to functionality is restricted based on user roles (Admin, Player, Referee), implemented via Spring Security annotations.

The system allows administrators to create and manage tournaments, players to register for tournaments and view their matches, and referees to manage match scores and results. The application features real-time notifications via WebSockets to keep users informed about important events.

2. Diagrams

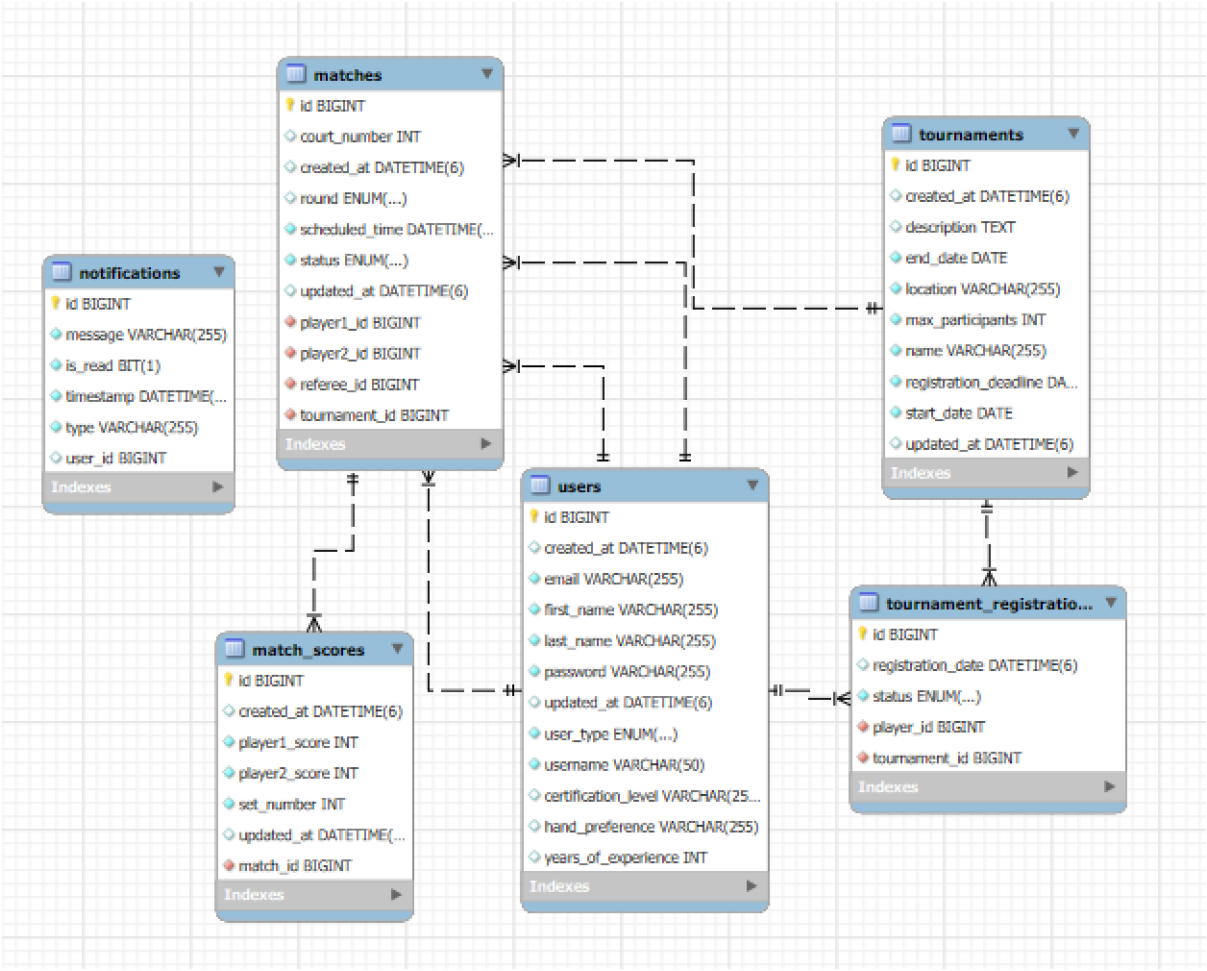
2.1 Database diagram

The database schema is designed to support the application's functionality while maintaining data integrity and relationships. The database consists of the following tables:

- users: stores user information and credentials
- tournaments: Stores tournament information
- tournament_registrations: maps players to tournaments with registration status
- matches: stores match information including players and referee
- match_scores: stores scores for each set in a match
- notifications: stores user notifications.

The database design includes several constraints to maintain data integrity:

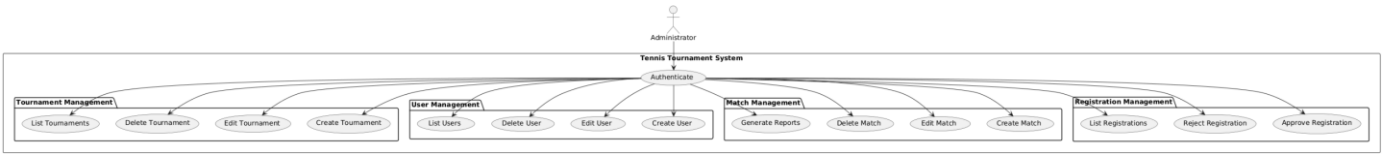
- Primary keys: each table has unique id
- Foreign keys: proper relationships between tables
- Unique constraints: prevents duplicate entries (unique usernames and emails in users table, unique set scores, unique tournament registrations)
- Not null constraints: required fields cannot be null (ex: username, user_type, etc).



2.2 Use case diagram

2.2.1 Admin use cases:

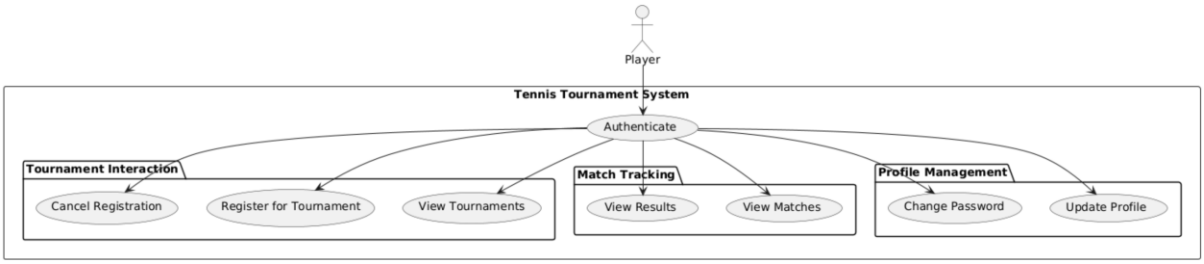
- create and manage tournaments
- manage user accounts
- organize matches between players
- approve/reject player registrations
- generate reports about match activities
- update personal information
- change password



2.2.1 Player use cases:

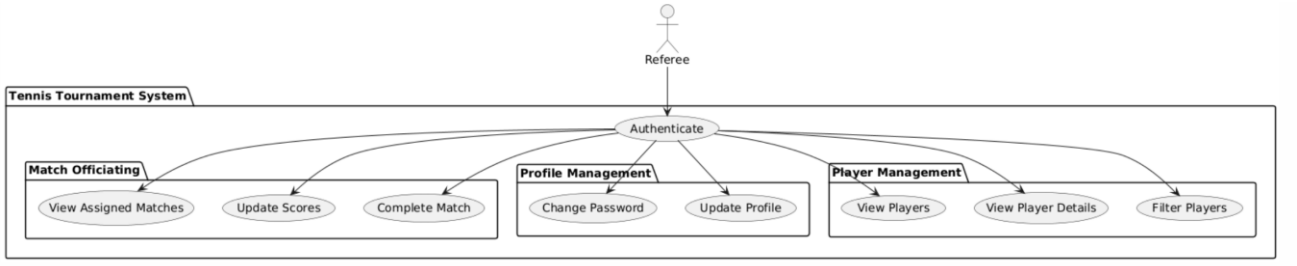
- view available tournaments
- register for tournaments

- view registration status
- view upcoming and past matches
- view match results and scores
- update personal information
- change password



2.2.3 Referee use cases:

- view assigned matches
- view players and player details
- filter players
- update match scores
- mark matches as completed
- update personal information
- change password



2.3 Class diagram

The Tennis Tournament Management System follows a well-structured layered architecture, which promotes separation of concerns, maintainability, and scalability. Below is a detailed description of how classes are organized within each layer and how these layers interact with each other.

1. **Presentation Layer**: this layer contains controllers that handle HTTP requests and provide REST API endpoints
    - AuthController: manages authentication endpoints (login, registration)
    - UserController: handles user management operations
    - TournamentController: exposes tournament management endpoints
    - TournamentRegistrationController: processes player registration requests
    - MatchController: manages match creation and updates
    - MatchScoreController: handles score recording and match completion
    - PlayerFilterController: REST Controller for filtering players with various criteria
    - ReportController: generates reports in various formats
    - NotificationController: manages user notifications

The controllers depend on the services in the Business Logic Layer and use DTOs to transfer data between the client and the application.

2. **Business Logic Layer**: this layer contains the core business logic and application services
   - **Services**:
     - UserService: implements user management and authentication logic
     - TournamentService: handles tournament management operations
     - TournamentRegistrationService: processes registration operations and waitlist management
     - MatchService: manages match operations and assignments
     - MatchScoreService: implements score recording, validation, and match completion
     - PlayerFilterService: service class for filtering players with various criteria
     - ReportService: creates reports in various formats
     - NotificationService: manages notification creation and delivery
     - EmailService: service for sending email notifications; uses Spring Mail with Thymeleaf templates for email content

   The application's services are tested using JUnit 5 for unit and integration tests, combined with Mockito for mocking dependencies.

   - **Security Components**:
     - JwtUtils: handles JWT (JSON Web Tokens) token generation and validation
     - AuthTokenFilter: processes authentication tokens for requests
     - WebSecurityConfig: configures security settings and access control
   - **Observer Pattern Components**:
     - Subject: interface for observable objects
     - Observer: interface for observer objects
     - MatchScoreSubject: notifies observers when match scores change
     - MatchScoreLogger: logs match score events
     - MatchScoreNotificationService: creates notifications based on score events
     - MatchScoreEvent: contains event data for score changes
   - **Builder Pattern Components**:
     - Builder: generic interface for builders
     - UserBuilder: creates User objects with validation
     - TournamentBuilder: creates Tournament objects with validation
     - MatchBuilder: creates Match objects with validation

The Business Logic Layer depends on the Data Access Layer for persistence operations and uses the Domain Model Layer for business entities. It transforms domain objects to DTOs for the Presentation Layer.

3. **Data Access Layer**: this layer provides access to the database and handles persistence operations
   - UserRepository: provides CRUD operations for User entities
   - TournamentRepository: manages Tournament entities
   - TournamentRegistrationRepository: handles TournamentRegistration entities
   - MatchRepository: provides access to Match entities

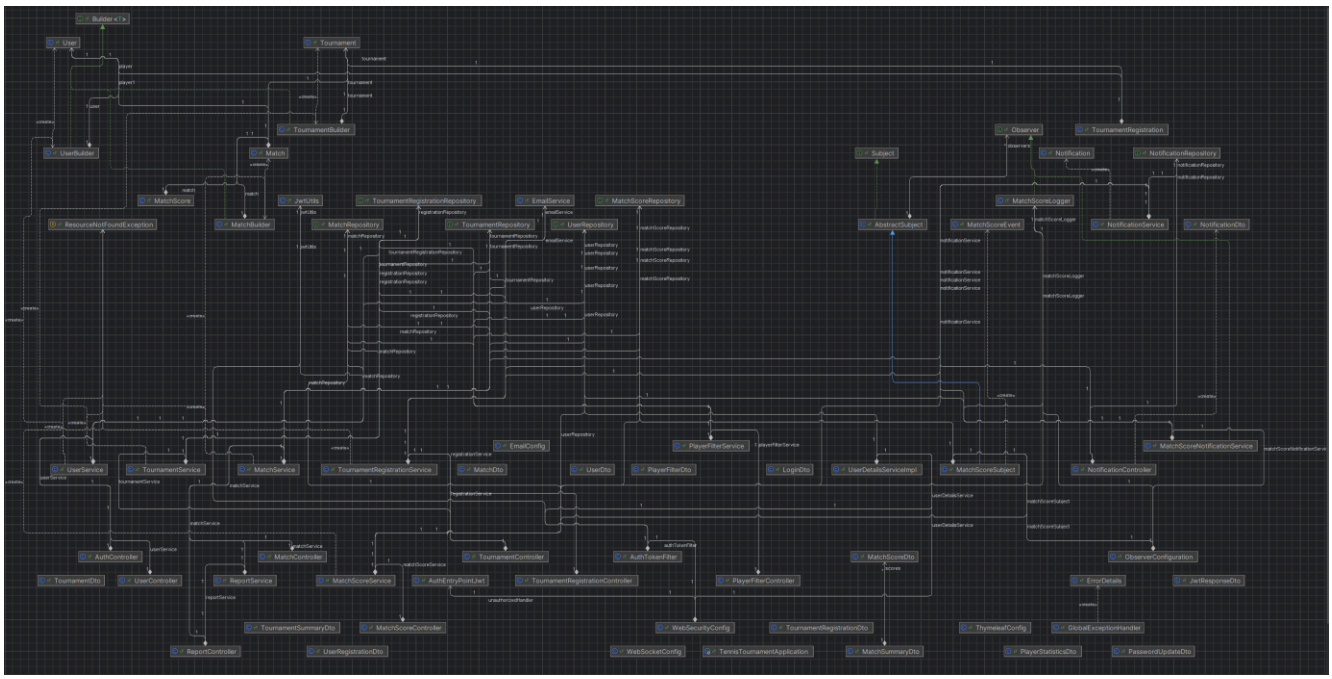
- MatchScoreRepository: manages MatchScore entities
- NotificationRepository: handles Notification entities

These repositories extend Spring Data JPA interfaces and provide custom query methods for specific data access needs. They depend on the Domain Model Layer for the entity definitions they manage.

4. **Domain Model Layer**: this layer contains the core business entities and value objects
    - User: represents system users with authentication information
    - Tournament: represents tennis tournaments
    - TournamentRegistration: links players to tournaments with status
    - Match: represents tennis matches between players
    - MatchScore: contains score information for match sets
    - Notification: represents system notifications

The Domain Model Layer forms the foundation of the application, with entities that encapsulate business rules and relationships.

5. **DTO Layer**: this layer contains Data Transfer Objects used for API communication. DTOs are used to decouple the domain model from the API representation, allowing independent evolution of both.



3. Conclusion

The Tennis Tournament Management System provides a comprehensive solution for organizing and managing tennis tournaments. Its layered architecture, clear separation of concerns, and use of established design patterns make it maintainable and extensible. The role-based access control ensures that users can only perform actions appropriate to their roles, while real-time notifications keep everyone informed about important events.

The relational database design efficiently supports the complex relationships between entities while maintaining data integrity. The combination of JPA/Hibernate for ORM, Spring Boot for the backend, and React for the frontend creates a robust and scalable application that meets the needs of tennis tournament organizers, players, and referees.